

SchnellPay – Phase 3.2: Withdrawal Delay + Broadcast Path (Design Doc)

****Status:**** DESIGN – no code written yet. For review before implementation.
****Context:**** Custodial withdrawals are currently DISABLED/inactive. This is a PRE-CUSTODIAL security gate – it must be built and tested before custodial goes live, but there is zero live risk in designing/building it now.
****Audience:**** Internal + external security reviewer.
****Author:**** Randeep (founderer)

0. How to read this / what we're asking the reviewer

This doc has three kinds of content, tagged inline:

- ****[EXISTS]**** – what's actually in the codebase today (verified by reading source, not assumed).
- ****[PLANNED]**** – what we intend to build in Phase 3.2.
- ****[REVIEW]**** – a specific question or decision where we want the reviewer's expertise.

****The honest framing for the reviewer:**** this is a **design** review. The security of a money-moving path lives largely in implementation details (race conditions, exact SQL, restart behavior) that a design doc can't fully capture. So the ask is:

****"Does this architecture have holes? Are we choosing the right mechanisms?"****

– NOT "is this secure?" (that requires reviewing the eventual code). A second review pass on the implementation is planned separately.

1. What SchnellPay is (so the design makes sense)

SchnellPay is a ****non-custodial**** quantum-safe crypto on-ramp TODAY. The custodial

feature set (holding user balances, processing withdrawals) is ****built in schema**

and partially in code but DISABLED** – it's gated behind a future money-transmitter

/ BaaS-partner licensing milestone. The intended custody model is ****via a licensed**

BaaS/CaaS partner, NOT DIY key custody** – i.e. SchnellPay will likely NOT hold raw

private keys or sign/broadcast transactions itself; a regulated partner does that.

- ****The existing payment "broadcast" is MOCKED**** - ``payment.service.ts`` uses ``generateMockTxHash()`` with comments "Simulate blockchain transaction / In production this would call actual blockchain APIs" and a ``setTimeout(5000)`` to "simulate confirmation." So no real on-chain broadcast exists anywhere yet.
- ****Custodial wallet addresses are demo-generated**** - ``custodial.service.ts`` has ``generateDemoAddress()`` with an explicit ``// replace with real KMS in production`` comment. Confirms custody/keys are not real yet (consistent with the BaaS-partner-not-DIY plan).

2.4 The existing (unsafe-for-money) deferral pattern [EXISTS]

```
`payment.service.ts:213`:
```js
setTimeout(() => { this.confirmPayment(paymentId); }, 5000); // simulate
confirmation
```
```

****[REVIEW]**** We consider in-process ``setTimeout`` UNSUITABLE for a money-moving withdrawal delay because it's lost on process restart, not persistent, not idempotent, and not observable. We plan a DB-backed approach instead (§4). Does the reviewer agree, or is there a case for a lighter mechanism at our scale?

3. What Phase 3.2 must achieve (security goals)

Phase 3.2 = ****enforce a delay between withdrawal approval and broadcast****, giving a window to detect and cancel fraudulent withdrawals (e.g. if an admin session is compromised and approves a malicious withdrawal). Concretely:

1. ****[PLANNED] A configurable delay**** (``WITHDRAWAL_DELAY_MINUTES``) between approval and actual broadcast.
2. ****[PLANNED] Fail-closed delay:**** if the delay config is missing, unreadable, or invalid, default to a LONGER safe delay – NEVER zero/immediate. (A misconfig must not silently disable the protection.)
3. ****[PLANNED] A cancellation window:**** during the delay, the withdrawal can be cancelled (by admin, and ideally by the user) before it broadcasts.
4. ****[PLANNED] An idempotent broadcast worker:**** something that, after the delay, moves eligible withdrawals to broadcast – and can NEVER double-broadcast the same withdrawal, even across crashes/restarts/concurrent runs.
5. ****[PLANNED] Safe state transitions:**** every transition guarded so a withdrawal can't be processed twice or processed from the wrong state.

This layers on top of Phase 2.1 (PIN-gated approval, already shipped) and will

later sit beneath Phase 3.1 (mandatory TOTP on withdrawal approval).

4. Proposed design

4.1 Delay representation [PLANNED]

Add an explicit column rather than computing on the fly:

...

```
ALTER TABLE custodial_transactions
  ADD COLUMN broadcast_after TIMESTAMP;  -- when this withdrawal becomes
  broadcast-eligible
...
```

On approval, set ``broadcast_after` = NOW() + (delay)``. The worker only picks up withdrawals where ``broadcast_after` <= NOW()``.

****[REVIEW]**** Explicit ``broadcast_after`` column vs. computing ``withdrawal_approved_at + delay`` at query time? We lean explicit (auditable, the delay-at-time-of-approval is frozen even if config changes later). Agree?

4.2 Fail-closed delay resolution [PLANNED]

...

```
resolveDelayMinutes():
  raw = featureFlag('WITHDRAWAL_DELAY_MINUTES') // or env
  if raw missing / not a positive integer / unparseable:
    log a warning
    return SAFE_DEFAULT_MINUTES // e.g. 60 – deliberately long, never 0
  if raw < FLOOR (e.g. 10): return FLOOR // can't be set dangerously
  low
  if raw > CEILING (e.g. 1440): return CEILING // sanity bound
  return raw
...
```

****[REVIEW]**** (a) Is "missing config → long default" the right fail-closed posture here, or should a missing config instead **block all broadcasts** (fail-stopped rather than fail-slow)? (b) Reasonable SAFE_DEFAULT / FLOOR / CEILING values for a small custodial platform? (c) Should the floor be enforced at write-time (reject a bad config) as well as read-time?

4.3 The broadcast worker – architecture options [PLANNED + REVIEW]

No worker infra exists, so we must choose one. Options:

****Option A – In-process interval (simplest):**** a ``setInterval`` in the existing Node backend that every N seconds queries eligible withdrawals and processes them.

- Pros: no new infra, deploys with existing backend.
- Cons: runs inside the web process; if we ever scale to >1 instance, multiple workers race (needs locking); dies if the process dies (Railway keeps it alive,

but mid-run crashes need idempotency).

****Option B – Separate Railway worker service:**** a dedicated process.

- Pros: isolated from web traffic; single-purpose; clearer failure domain.
- Cons: new service to deploy/monitor; still needs idempotency + single-runner guarantee.

****Option C – External scheduler (Railway cron / pg_cron) hitting an internal endpoint.****

- Pros: simple, stateless trigger.
- Cons: the endpoint must be locked-down + idempotent; another moving part.

****[REVIEW]**** For a solo-founder, single-Railway-instance, low-volume custodial platform at launch – which architecture, and what's the migration path when volume grows? We currently lean ****Option A with a DB-level lock (§4.4)**** for launch, with a documented path to Option B at scale. Is that defensible, or is starting with B worth the overhead?

4.4 Idempotency + single-runner safety [PLANNED – the crux]

The core money-safety property: ****a withdrawal must broadcast at most once.****
Plan:

- Worker claims a row with an atomic conditional UPDATE (optimistic lock), same pattern we used in Phase 2.1's rowCount guard:

```
```sql
UPDATE custodial_transactions
 SET status = 'broadcasting', updated_at = NOW()
 WHERE id = $1 AND status = 'processing' AND broadcast_after <= NOW()
-- rowCount == 1 → we own it; rowCount == 0 → someone else took it / not
 eligible
```
```

Only the runner that flips `processing → broadcasting` (rowCount=1) proceeds.

- Use `SELECT ... FOR UPDATE SKIP LOCKED` when batch-claiming, so concurrent workers never grab the same row.
- The actual broadcast call (partner API) must itself be idempotent or use an idempotency key, in case we crash AFTER broadcasting but BEFORE recording success.

****[REVIEW]**** This is the highest-stakes part. Is the conditional-UPDATE claim + `FOR UPDATE SKIP LOCKED` sufficient, or do we need a stronger guarantee (e.g. a dedicated `broadcast\_attempts` ledger / outbox pattern)? ****Specifically: what happens if the partner broadcast succeeds but our DB write of `completed` fails?****

How should we reconcile (the classic dual-write problem)? This is our biggest worry.

4.5 Cancellation during the window [PLANNED]

While `status='processing'` and `broadcast\_after > NOW()`, allow cancellation:

```
```sql
UPDATE custodial_transactions
```

```
SET status='cancelled', notes=$2, updated_at=NOW()
WHERE id=$1 AND status='processing'
``,``
```

- Admin cancellation: a PIN-gated admin endpoint (reuse Phase 2.1 `checkAdminPin`).
- **[REVIEW]** Should the *\*user\** also be able to cancel their own pending withdrawal during the window (good UX + extra fraud safety), and does that introduce any abuse vector (e.g. griefing, race with the worker)?
- Race: cancellation vs. the worker claiming the row – the conditional UPDATE in §4.4 (`WHERE status='processing'`) means whichever fires first wins atomically; the loser gets rowCount=0. **[REVIEW]** confirm this is race-safe.

### ### 4.6 What we will NOT build now (the partner stub) [PLANNED]

The actual on-chain broadcast / partner API call is a **\*stub\*** that just logs and sets a mock tx\_hash – because the custody partner isn't chosen. The state machine, delay, idempotency, and cancellation are all built and testable around the stub.

**[REVIEW]** Is stubbing the broadcast while building the full surrounding control flow a sound approach, or does the partner integration shape the control flow enough that we should pin the partner first?

---

### ## 5. Threat model (what this defends against)

Threat	Mitigation in 3.2
Compromised admin session approves a fraudulent withdrawal	Delay window + cancellation; (PIN from 2.1; TOTP from 3.1)
Double-broadcast (user paid twice)	Idempotent claim (§4.4)
Misconfigured/disabled delay silently removes protection	Fail-closed delay (§4.2)
Worker crash mid-broadcast	Idempotency + reconciliation (§4.4) – <b>*the open question*</b>
Withdrawal processed from wrong state	Conditional-UPDATE state guards (§4.4, §4.5)
Race between cancel and broadcast	Atomic single-winner UPDATE (§4.5)

**[REVIEW]** What threats are we MISSING? Particularly around the partner boundary, the dual-write problem, and anything specific to crypto withdrawals (e.g. address substitution, amount tampering between approval and broadcast – should we re-verify the withdrawal address/amount at broadcast time against what was approved?).

---

## ## 6. Specific asks for the reviewer (summary)

1. **\*\*§4.4 dual-write / idempotency\*\*** – our biggest concern. Is conditional-UPDATE + SKIP LOCKED enough? How to handle "broadcast succeeded, DB write failed"? Is an outbox/ledger pattern warranted at our scale?
2. **\*\*§4.2 fail-closed posture\*\*** – fail-slow (long default) vs. fail-stopped (block all broadcasts) on missing/bad config? Sensible bounds?
3. **\*\*§4.3 worker architecture\*\*** – in-process interval + DB lock for launch, migrate to separate worker at scale – defensible?
4. **\*\*§4.5 cancellation\*\*** – allow user self-cancel in the window? Abuse vectors?
5. **\*\*§5 threat model gaps\*\*** – what are we not seeing, especially re-verifying address/amount at broadcast time, and partner-boundary threats?
6. **\*\*§4.6 stub approach\*\*** – build control flow now with broadcast stubbed, or pin the custody partner first?

---

- ## ## 7. What is explicitly OUT of scope for 3.2 (so review stays focused)
- Phase 3.1 (mandatory TOTP on approval) – separate, layered on later.
  - Choice of custody/BaaS partner – business/legal track, not this doc.
  - Real KMS / key handling – partner's responsibility under the intended model.
  - Sanctions/address screening (`screening\_status` exists in schema) – related but a separate workstream (Chainalysis/TRM-class provider, gated on volume/licensing).
  - Confirmation tracking (`confirmations`/`required\_confirmations`) – post-broadcast monitoring, a follow-on to this.

---

## ## 8. Build order once design is approved

1. Migration: add `broadcast\_after` column (+ any index for the worker query).
2. `resolveDelayMinutes()` fail-closed helper (+ tests).
3. Wire approval to set `broadcast\_after` (modify the Phase 2.1 `approveWithdrawal`).
4. Broadcast worker (chosen architecture) with conditional-claim idempotency + **\*\*stubbed\*\*** partner broadcast.
5. Cancellation endpoint(s) (PIN-gated admin; maybe user self-cancel).
6. Tests: delay enforced, fail-closed on bad config, no double-broadcast under concurrency, cancel-vs-broadcast race, restart safety.
7. (Later, at partner selection) replace the broadcast stub with the real partner call + idempotency key + reconciliation.

---

\*Prepared for design review. Implementation review to follow separately on the actual code. Nothing in this doc is live; custodial withdrawals remain disabled.\*